can数据帧utc时间戳修改cpu单调时间戳

1.背景

```
1:utc时间受外界影响较大,比如在机器运行过程中utc时间受上位机影响较大,有概率导致整个系统异常。
```

2.切换时间戳为机器开机单调时间

1. 实现原理

```
socketcan 协议在启用 SO_TIMESTAMP 标志位后
        setsockopt(fd, SOL_SOCKET, SO_TIMESTAMP, &en, sizeof(en));

通过接口可以获取到当前帧的utc时间戳
    recvmsg(fd, &msg, MSG_DONTWAIT);
    const struct cmsghdr* cmsg = CMSG_FIRSTHDR(&msg);

通过内核源码中添加新flag,在应用层设置对应flag后,消息帧时间戳修改为cpu单调是时间戳
```

2. 应用层修改

```
///添加新的flag
#define SO_KVIOLET_MONOTONIC_TIMESTAMP 80 /// enable kernel cpu monotinic ts const int en = 1;
setsockopt(fd, SOL_SOCKET, SO_KVIOLET_MONOTONIC_TIMESTAMP ,&en, sizeof(en));
```

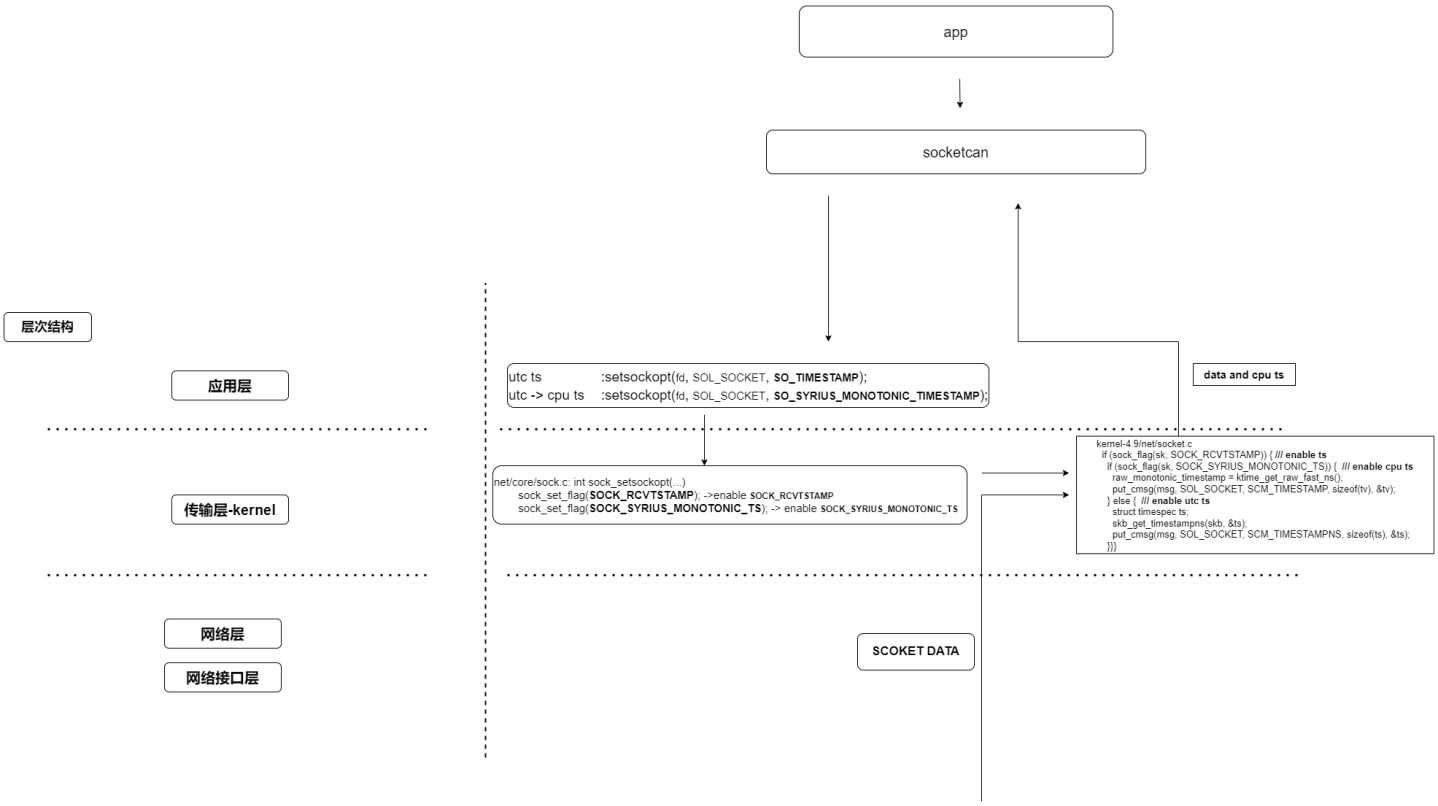
3. 内核修改

```
/// 接收应用层setsockopt指令
case SO_KVIOLET_MONOTONIC_TIMESTAMP :
        pr_warn("syrius setsockopt:en:%d,opt:%d \n", valbool, optname);
        if (valbool){
                sock_set_flag(sk, SOCK_SYRIUS_MONOTONIC_TS);
        }else{
```

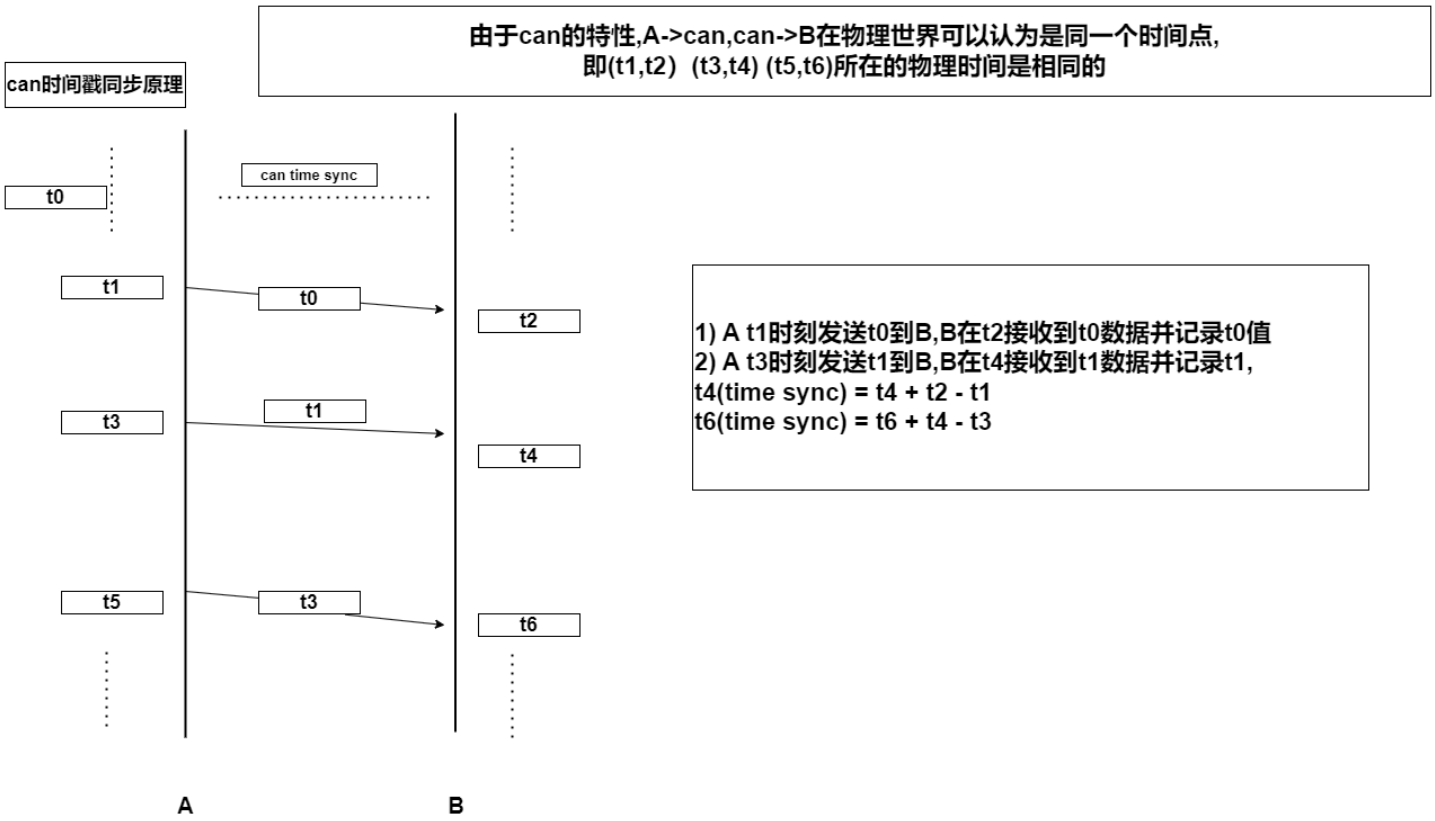
```
                sock_reset_flag(sk, SOCK_SYRIUS_MONOTONIC_TS);
        }
        break;


/// 消息帧修改时间为单调时间
/// ktime_get_raw_fast_ns 内核版本不同调用接口也不进相同
if (sock_flag(sk, SOCK_SYRIUS_MONOTONIC_TS)){
        raw_monotonic_timestamp = ktime_get_raw_fast_ns();
        struct timespec64 tts64 = ns_to_timespec64(raw_monotonic_timestamp);
        if (!sock_flag(sk, SOCK_RCVTSTAMPNS)){
                struct timeval tv;
                tv.tv_sec = tts64.tv_sec;
                tv.tv_usec = tts64.tv_nsec / 1000;
                put_cmsg(msg, SOL_SOCKET, SCM_TIMESTAMP,sizeof(tv), &tv);
        }else{
                struct timespec ts;
                ts.tv_sec = tts64.tv_sec;
                ts.tv_nsec = tts64.tv_nsec;
                put_cmsg(msg, SOL_SOCKET, SCM_TIMESTAMPNS,sizeof(ts), &ts);
        }
}
```

4. 层次结构



3.can时间戳同步原理



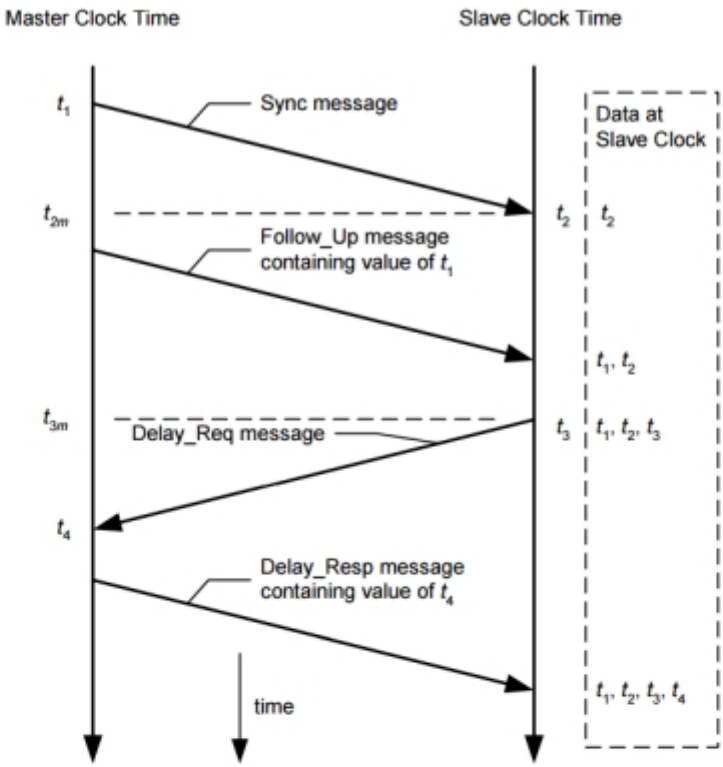


Figure 2.13: PTP message exchange diagram [tutorial]

Afterwards, the slave sends to the master a *Delay_Req* message, recording its sent time (t_3) and then waits for the master's response containing the time in which it received it (t_4).

With all the time references acquired, the slave can estimate the offset value with expression 2.6 and the communication delay with the expression 2.7. The derivation of these expressions can be found in [35].

$$offset = \frac{(t_2 - t_1) - (t_4 - t_3)}{2} \tag{2.6}$$

$$delay = \frac{(t_2 - t_1) + (t_4 - t_3)}{2} \tag{2.7}$$

Knowing the offset of the clocks, the slave can correct its local clock to be synchronized with the master, but the IEEE 1588 standard says nothing about how to do this.